

Advanced Chatbot Documentation (Deep Explanation Style)

app.py - Deep Explanation

This file is responsible for creating the user interface of the chatbot using Streamlit. It acts as the bridge between the user and the AI model.

The line 'import streamlit as st' imports Streamlit, which is a library used to build interactive web applications using Python without needing frontend technologies like HTML or JavaScript.

The line 'from model import chatbot_response, learn' imports two important functions from model.py. The chatbot_response function is used to generate replies, while the learn function allows the chatbot to store new knowledge dynamically.

The command 'st.title()' sets the title of the web application. This improves the user interface and gives the application a professional appearance.

The session_state is used to store chat history. Normally, Streamlit reruns the script on every input, which would erase previous messages. To solve this, session_state keeps the chat persistent.

The 'st.text_input()' creates an input box where the user types their message. This value is stored in the variable user_input.

When the user enters text, the chatbot_response function is called. This function processes the input and tries to find the most relevant answer using NLP techniques.

If a response is found, both the user message and bot reply are stored in session_state.chat. This ensures that the conversation history is preserved and displayed.

If no response is found, the chatbot asks the user to teach it. This is the self-learning feature of the chatbot.

The new answer provided by the user is passed to the learn function, which saves it into the knowledge base file. This allows the chatbot to improve over time.

Finally, a loop is used to display all chat messages stored in session_state, creating a continuous conversation view.

model.py - Deep Explanation

This file acts as the brain of the chatbot. It handles text processing, similarity matching, and learning new responses.

The json library is used to read and write data from the knowledge.json file, which stores all chatbot knowledge.

TfidfVectorizer converts text into numerical vectors. This is necessary because machine learning models cannot understand raw text directly.

Cosine similarity is used to measure how similar two sentences are. It compares the angle between vectors to determine similarity.

The knowledge base is loaded from knowledge.json using UTF-8 encoding to support special characters and emojis.

All patterns (user questions) are extracted and stored in a list. Corresponding answers are stored in another list. This creates a mapping between questions and answers.

The vectorizer is trained on all patterns. This converts all stored questions into a mathematical representation.

When a user enters a query, it is transformed into a vector and compared with all stored vectors using cosine similarity.

The system finds the most similar question and returns the corresponding answer along with a similarity score.

If the similarity score is above a certain threshold, the chatbot returns the answer. Otherwise, it returns None, indicating it doesn't know the answer.

The learn function allows the chatbot to add new knowledge. It appends new patterns and answers to the knowledge base and saves them into the JSON file.

The vectorizer is retrained dynamically after learning, allowing the chatbot to use new knowledge immediately without restarting.

knowledge.json - Deep Explanation

This file acts as the memory of the chatbot. It stores all known questions and answers in a structured format.

Each entry contains 'patterns', which are different ways a user might ask the same question, and an 'answer', which is the response returned by the chatbot.

Using multiple patterns improves accuracy because users can ask the same question in different ways.

As the chatbot learns from users, new entries are added to this file, making the system smarter over time.